
Public Scheme Eligibility Assistant

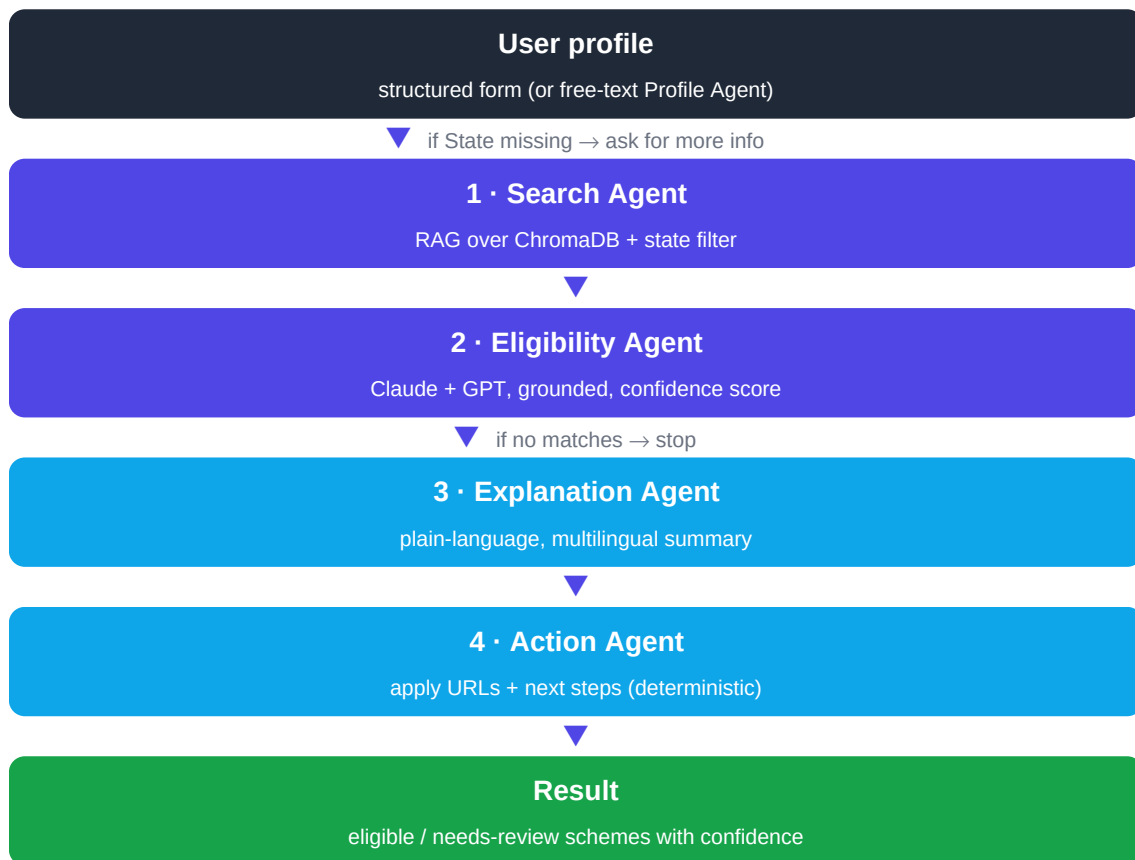
An agentic AI system that matches Indian citizens to the government welfare schemes they qualify for.

Multi-LLM (Claude + GPT) · RAG over ~4,700 schemes · LangGraph orchestration

Problem	Citizens miss welfare schemes because eligibility rules are hard to parse.
Solution	Ask a few questions, retrieve relevant schemes, check eligibility with two LLMs in agreement, explain in plain language + local language, and point to how to apply.
Humans in control	Low-confidence or ambiguous cases are flagged for review, never guessed.

Architecture — a LangGraph agent pipeline

Five agents are orchestrated as a LangGraph **StateGraph**. Typed state flows between nodes, and two **conditional gates** route the flow: a profile-completeness check and a no-results check.



Defined in `graph.py`; each node wraps one agent in `agents/`. A Profile Agent (Claude + GPT) also exists for free-text intake.

The agents

Agent	Role	Approach
Profile	Turn free text into a structured profile	Claude + GPT ensemble, Pydantic
Search	Find candidate schemes for the profile	Semantic RAG (Chroma) + state metadata filter
Eligibility	Judge profile vs each scheme's rules; list documents	Both LLMs, grounded in scheme text, confidence-scored
Explanation	Friendly summary in the user's language	Single LLM, grounded in the verdicts
Action	Apply URLs + next steps	Deterministic extraction from the data

Multi-LLM confidence engine

The core innovation: every eligibility decision is made by **two independent models** (Anthropic Claude + OpenAI GPT) at temperature 0. Their agreement becomes a confidence score.

- **Both agree** → high confidence (100%), verdict trusted.
- **They split** → 50%, verdict marked **unclear** and flagged **needs review**.
- **Either is unsure** → verdict is **unclear** (the system refuses to guess).
- Adding a third model (e.g. Gemini) is one new adapter — the interface is provider-agnostic.

This is the human-in-the-loop guardrail: on a welfare tool, a flagged “maybe” is safer than a confident wrong answer.

Retrieval (RAG) design

- ~4,700 Indian schemes; each scheme's text is split into **overlapping chunks** (180 words, 40-word overlap) → ~19,000 chunks.
- Chunking is necessary because the embedding model (all-MiniLM-L6-v2) only reads ~256 tokens — without it, most of each scheme would never be indexed.
- Embeddings stored in **ChromaDB**; retrieval collapses chunks back to unique schemes.
- **Hybrid filter**: results are limited to the user's state + nationally-available (“All”) schemes, so matches are both relevant and geographically valid.
- Eligibility is **grounded** — the model judges only against the scheme's own eligibility text and never invents criteria.

Technology stack

Python	Streamlit	LangGraph	Anthropic Claude
OpenAI GPT	ChromaDB	Sentence-Transformers	Pydantic

Status & next steps

Working end to end

- Full pipeline (search → eligibility → explanation → action) live in a Streamlit UI.
- Multi-LLM confidence + needs-review flagging.
- LangGraph orchestration with conditional routing.
- Eval harness with golden test profiles; graph exportable as a diagram.

Planned

- True LangGraph human-in-the-loop interrupt (pause & resume).
- Cloud deployment (Railway) for a public URL.
- Structured rule extraction to make eligibility fully deterministic + auditable.

This tool provides guidance only. Users should confirm details on the official government portal before applying.